



ADO.NET

Manipulation des Données en C#

Entity Framework Core

Entity Framework Core est ce qu'on appelle un ORM (Object Relational Mapping) : il va se charger pour nous de transcrire les changements au niveau de notre code en requête SQL pour mettre à jour la Base de données.

Pour utiliser EF Core, il faut installer plusieurs packages Nuget, qui sont principalement regroupé en trois grosses séries de packages :

- **Microsoft.EntityFrameworkCore.Tools** pour avoir accès aux lignes de commandes dans la console de gestionnaire de package
- **Microsoft.EntityFrameworkCore.Design** pour permettre le build et la conception des migrations basées sur nos modèles
- **Microsoft.EntityFrameworkCore.SqlServer** pour utiliser EF Core en relation avec une Base de données SQL Server

Puis, il nous faudra réaliser une injection de dépendance basée sur une classe que l'on nommera par convention **ApplicationDbContext**, et qui héritera de la classe **DbContext** afin d'en posséder les méthodes.

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
}
```

AddDbContext<>()

Pour utiliser EF Core, il faudra également injecter ce fameux ApplicationDbContext au niveau d'une application de type ASP.NET

Contrairement aux injections standards, EF Core nous apporte une extension de méthode pour nos services qui se nomme AddDbContext<T> où T est la classe nous servant de contexte de Base de données. Nous pouvons ensuite le paramétrer comme nous le faisons dans l'exemple ci-dessous dans le but de spécifier l'utilisation de SQL Server et d'une chaîne de connexion obtenue depuis le fichier **appsettings.json** ou **secrets.json**

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
```

De son côté, l'**ApplicationDbContext** ne contiendra généralement que des propriétés publiques de type **DbSet<T>** où T est la classe que l'on souhaite mapper, comme par exemple ici nos **chiens**, nos **maîtres** et nos **adresses**.

```
6 references  
public DbSet<Dog> Dogs { get; set; }  
6 references  
public DbSet<Master> Masters { get; set; }  
6 references  
public DbSet<Adress> Adresses { get; set; }
```

Le nom que l'on donne à ces DbSet sera utilisé par EF Core lors de la création d'une migration pour peupler la Base de données de ces modèles.

La Migration

Une fois l'ApplicationDbContext configuré et nos modèles créés, il faut réaliser ce qu'on appelle une migration dans le but de faire se retranscrire notre code sous la forme d'une base de données. Pour cela, rien de plus simple, il suffit d'ouvrir la console de gestionnaire de packages et d'entrer la commande **Add-Migration <nomDeMigration>**.

```
Package Manager Console Host Version 6.1.0.106  
  
Type 'get-help NuGet' to see all available NuGet commands.  
  
PM> Add-Migration Blabla_my_Migration
```

Le nom de la migration sera retranscrit sous la forme d'un fichier stocké dans un dossier Migrations qui se trouvera à la racine de votre projet et qui contiendra toutes les migrations que vous pouvez avoir eu envie de réaliser. De plus, par convention, la première migration se nommera **Initial**.

Dans le cas d'une erreur lors de la modification du script de migration, il suffit d'utiliser la commande Remove-Migration pour annuler la migration.

Une fois la migration personnalisée et validée par vos soins, il faut modifier la base de donnée. Pour cela, on se sert de la commande **Update-Database** (à savoir qu'il est possible de passer à une migration particulière en ajoutant la **référence** de cette dernière en paramètre de Update-Database => **Update-Database <nomDeMigration>**).

```
PM> Update-Database 20220315172509_Blabla_my_Migration
```

Manipuler les données

Une fois la migration faite, il est possible de passer à la construction de notre application. Pour modifier les données, il suffira de manipuler les **DbSets** comme on le ferait avec des listes tout en n'oubliant pas de faire appel à la méthode **SaveChanges()** sur votre **ApplicationDbContext** lorsque vous souhaitez modifier les valeurs en Base de données.

```
2 references
public Dog Add(Dog entity)
{
    _context.Dogs.Add(entity);

    _context.SaveChanges();
}
```

En cas de nécessité d'agregation de type **One-to-Many** ou **Many-to-Many**, il vous faudra modifier vos modèles pour prendre en compte ces possibilités, comme ci-dessous dans le cadre d'une relation de type One-to-Many :

```
8 references
public int MasterId { get; set; }

[ForeignKey("MasterId")]
5 references
public virtual Master? Master { get; set; }
```

Le mot-clé **virtual** sert à EF Core lorsqu'il a besoin de surcharger l'accès à cette propriété (qui s'appelle une propriété de navigation). L'ajout de l'annotation **ForeignKey** n'est pas obligatoire ici, car EF Core sait repérer les noms de propriétés en liens avec les clés de votre modèle, comme par exemple Id, ModelId, ClassId, etc...

Les propriétés de navigation

Les **propriétés de navigation** sont extrêmement utiles lorsque l'on cherche à remplir les champs d'un modèle, comme ici lorsqu'on cherche à alimenter la valeur de **Master** dans le modèle **Dog**.

```
return _context.Dogs
    .Include(x => x.Master).ThenInclude(x => x.Address)
    .FirstOrDefault(x => x == entity);
```

Via l'utilisation des méthodes *Include()* puis *ThenInclude()*, il est possible d'alimenter notre modèle Dog des valeurs du Maître, qui sera lui-même alimenté de son adresse via l'utilisation de multiples **INNER JOIN**. Dans le cas où l'on souhaite voir la magie derrière le rideau, sachez qu'il est possible d'afficher les **requêtes SQL** qu'EF Core génère pour vous via le paramétrage de **ApplicationDbContext** :

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("Default"))
    .LogTo(Console.WriteLine, LogLevel.Information));
```

```
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (32ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [d].[Id], [d].[Age], [d].[CollarColor], [d].[MasterId], [d].[Name], [m].[Id], [m].[AddressId], [m].[Email],
[m].[Firstname], [m].[Lastname], [m].[Phone], [a].[Id], [a].[CityName], [a].[PostalCode], [a].[StreetName], [a].[StreetN
umber]
      FROM [Dogs] AS [d]
      INNER JOIN [Masters] AS [m] ON [d].[MasterId] = [m].[Id]
      INNER JOIN [Addresses] AS [a] ON [m].[AddressId] = [a].[Id]
info: 15/03/2022 21:27:49.522 RelationalEventId.CommandExecuted[20101] (Microsoft.EntityFrameworkCore.Database.Command)
      Executed DbCommand (32ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [d].[Id], [d].[Age], [d].[CollarColor], [d].[MasterId], [d].[Name], [m].[Id], [m].[AddressId], [m].[Email],
[m].[Firstname], [m].[Lastname], [m].[Phone], [a].[Id], [a].[CityName], [a].[PostalCode], [a].[StreetName], [a].[StreetN
umber]
      FROM [Dogs] AS [d]
      INNER JOIN [Masters] AS [m] ON [d].[MasterId] = [m].[Id]
      INNER JOIN [Addresses] AS [a] ON [m].[AddressId] = [a].[Id]
```